
Machine Learning Based Analog Circuit Fault Diagnosis
PDF 3 — Chapters 7-8: Dataset Generation & Data Preprocessing
How the Dataset Was Built | Interpolation, Feature Matrix, Scaling, Train-Test Split

CHAPTER 7 — Dataset Generation

Chapter 7 — Dataset Generation (How We Created the Data) Before Starting In the previous chapter we learned how LTspice generates waveforms. Now a question arises. Question. One waveform is enough for Machine Learning? No.

Machine Learning

needs hundreds or thousands of examples. So, how did we generate our dataset? That is this chapter.

What you will know after finishing this chapter

After finishing this chapter you should be able to answer [OK] What is a dataset? [OK] Why is a dataset needed? [OK] How was our dataset generated? [OK] What is fault injection? [OK] Why labeling is important? [OK] Why supervised learning requires datasets? [OK] Explain dataset generation in viva.

Difficulty

Basics 5/5

Theory 5/5

Coding 1/5

Viva Importance 5/5

Let's Begin

What is a Dataset? Suppose I ask you to teach a child how to identify apples. Question. Will showing one apple be enough? No. Need many apples. Green apples. Red apples. Small apples. Big apples. Only then the child learns.

Machine Learning

works the same way. Instead of apples, we show waveforms.

Simple Definition

A dataset is simply a collection of examples used for training and testing a Machine Learning model.

PROFESSOR ASKS

What is a dataset?

EXCELLENT ANSWER

A dataset is an organized collection of labeled samples that is used to train and evaluate a machine learning model. Remember Collection of Labeled Samples Why Do We Need a Dataset? Question. Can SVM learn without examples? No. Imagine trying to teach someone mathematics without any questions. Impossible. Similarly

Machine Learning

learns from examples. Question. What are our examples? Waveforms.

Our Dataset

Our dataset contains many waveforms. Each waveform belongs to one class. Example

Waveform 1

RC_Normal **Waveform 2** — RC_Open. **Waveform 3** — RL_Open. **Waveform 4** — RLC_Normal. and so on. Question. What do we call these names? Labels. Very important.

PROFESSOR ASKS

What is a Label?

EXCELLENT ANSWER

A label is the correct output associated with a training sample. In our project, the label represents the actual fault class of the waveform.

One Sample

Let's see what one sample actually means. One sample contains

Voltage Waveform

+

Correct Fault Name

That's all.

Machine Learning

learns from thousands of such samples. How Did We Generate the Dataset? Now the most important question. Question. Did we download a dataset? No. We built it. That is one of the biggest strengths of your project. Our process looked like this.

Design Circuit

Healthy Circuit

Run Simulation

Store Waveform

Again

Same Circuit

Introduce Fault

Run Simulation

Store Waveform

Again Another Fault. Again. Another Fault. Again. Another Circuit. Eventually we get hundreds of waveforms. Question. Why repeat the simulation so many times? Because every fault produces a different waveform.

Healthy Sample

Example **Circuit** — Normal. **Simulation** — Waveform. **Label** — RC_Normal. Faulty Sample Example Circuit

Resistor Open

Simulation — Waveform. **Label** — RC_Open. Notice only one thing changed. The fault. Everything else remains the same.

Fault Injection

This is one of the most important concepts. Question. What is Fault Injection? Simple. We intentionally introduce a fault into the circuit. Notice the word Intentionally. The fault didn't happen randomly. We created it on purpose.

PROFESSOR ASKS

What is Fault Injection?

EXCELLENT ANSWER

Fault injection is the deliberate introduction of known faults into a circuit so that the resulting behavior can be studied and correctly labeled. Excellent. Why Do We Inject Faults? Question. Suppose a

waveform changes. Question. How do we know which fault caused it? We know because we created the fault ourselves. That means our labels are reliable.

PROFESSOR ASKS

How do you know your labels are correct?

EXCELLENT ANSWER

Since each fault condition was intentionally introduced before simulation, the corresponding waveform label is known with certainty. Very important. Types of Samples Our dataset contains two major types.

Healthy Samples

Everything works. Correct waveform. Correct behavior.

Faulty Samples

One component has been modified. Waveform changes. Different label.

Machine Learning

learns the difference between them. Why Not Collect Hardware Data? Question. Could we collect real hardware data? Yes. Question. Why didn't we? Let's compare. **Hardware** — Need. components. Need oscilloscope. Need fault creation. Need measurements. Very slow. **LTspice** — Click. Simulate. Export. Repeat. Much faster.

PROFESSOR ASKS

Why simulation instead of hardware?

EXCELLENT ANSWER

Simulation provides a controlled environment where faults can be introduced systematically, allowing rapid generation of accurately labeled datasets. Excellent. Is Simulation Perfect? No. Very important. Simulation does not include everything. Example Real hardware has Noise. Temperature effects. Manufacturing variations. Parasitic capacitances. Simulation may ignore some of these.

PROFESSOR ASKS

Then why trust the dataset?

EXCELLENT ANSWER

Although simulation cannot perfectly represent every real-world effect, it provides consistent and accurately labeled data, making it highly suitable for developing and validating the proposed methodology.

One Important Observation

Notice something. Our dataset is not random. Every sample has meaning. Example RC_Normal

Known Circuit

Known Components

Known Waveform

Known Label

Everything is controlled. That is a high-quality dataset. What Happens After Dataset Generation? Question. Now we have hundreds of waveforms. Can SVM use them? No. Question. Why? Because they are still raw waveforms. Need Preprocessing. That is our next chapter. Complete Dataset

Generation Flow Circuit Design

Healthy Circuit

Fault Injection

LTspice Simulation

Voltage Waveform

Assign Label

Store Sample

Repeat

Final Dataset

Memorize this.

Connection to Our Project

Now connect everything. **Circuit** — Simulation.

Fault Injection

Waveform — Label. Dataset

Machine Learning

Notice

Machine Learning

has still not started. We are only preparing the data. This is why I kept telling you that 80% of Machine Learning is actually data preparation.

One Important Lesson

Many beginners think

Machine Learning

starts here. Dataset

Train Model

Wrong. In our project

Machine Learning

starts much later. Before that we still have Interpolation. Preprocessing. Feature Extraction. Scaling. Only then Machine Learning. Remember this.

COMMON MISTAKES

MISTAKE: [X] **Dataset** — Random waveforms. Wrong. Every waveform has a known label.

MISTAKE: [X] Faults occurred naturally. Wrong. We intentionally introduced them.

MISTAKE: [X] Simulation is perfect. Wrong. Simulation approximates real circuits.

MISTAKE: [X] Machine Learning starts after LTspice. Wrong. Several preprocessing steps still remain.

REVISION NOTES

Remember
these six points.

Dataset — Collection of labeled samples.

Sample — Waveform + Label.

Fault Injection — Intentionally creating faults.

Simulation — Generates waveforms.

Label — Correct fault class.

Dataset — Foundation of supervised learning.

MEMORY TRICK

Remember $C \rightarrow F \rightarrow S \rightarrow L \rightarrow R$ Expand it. $C = \text{Circuit}$ $F = \text{Fault Injection}$ $S = \text{Simulation}$ $L = \text{Label}$ $R = \text{Repeat}$ Repeat until the dataset is complete. Whenever someone asks "How was the dataset generated?" Just remember $C \rightarrow F \rightarrow S \rightarrow L \rightarrow R$

KNOWLEDGE CHECK

Level 1

- What is a dataset?
- What is one sample?
- What is a label?
- What is fault injection?
- Why do we need a dataset?

Level 2

- Why is labeling important?
- Why do we intentionally create faults?
- Why is simulation useful for dataset generation?
- Why can't we directly train SVM after generating the dataset?
- Why is our dataset considered reliable?

Professor Level

- How does your dataset support supervised learning?
- What are the limitations of simulation-generated datasets?
- How would you extend this dataset using real hardware?
- What challenges would arise if fault labels were incorrect?
- Why is data quality more important than simply increasing the number of samples?
- End of Chapter 7

Congratulations. You now understand how the dataset was created, why it is reliable, and why dataset generation is one of the strongest parts of your project.

Author's Note (Important) I noticed something while writing these first seven chapters. Originally, I planned for about 70 chapters, but I think that would make some chapters too short and split related ideas unnecessarily. Instead, I'm going to organize the Project Bible into larger, more complete chapters. For example, instead of having separate chapters for "Train-Test Split," "StandardScaler," and "Label Encoding," I'll combine them into a comprehensive Data Preprocessing chapter. This will make the Bible: easier to study, easier to revise before a viva, and much closer to how we actually learned the project together. So from the next chapter onward, you'll notice each chapter becomes deeper and more complete, covering an entire topic from basics to implementation rather than breaking it into many tiny sections. I think this will produce a much better final Project Bible.

CHAPTER 8 — Data Preprocessing

Chapter 8 — Data Preprocessing (Preparing the Dataset for Machine Learning) Before Starting At the end of Chapter 7, our dataset looks like this. **LTspice** — CSV Files.

Voltage Waveforms

Labels Question. Can we train SVM now? Most beginners say Yes. Wrong. Machine Learning models are very sensitive to the quality of input data. Raw data is almost never used directly. Need Preprocessing.

What you will know after finishing this chapter

After finishing this chapter, you should be able to answer [OK] What is preprocessing? [OK] Why is preprocessing necessary? [OK] Why can't SVM directly use the dataset? [OK] What preprocessing did we actually perform? [OK] Why interpolation? [OK] Why scaling? [OK] Why train-test split? [OK] Why label encoding?

Difficulty

Basics 5/5

Theory 5/5

Coding 4/5

Viva Importance 5/5

Let's Begin

What is Preprocessing? Simple definition. Preprocessing means preparing raw data before giving it to Machine Learning. Think of cooking. Question. Can you directly cook vegetables without washing or cutting them? No. Need Preparation. Exactly the same.

Raw Dataset

Preprocessing

Machine Learning

PROFESSOR ASKS

What is data preprocessing?

EXCELLENT ANSWER

Data preprocessing is the process of transforming raw data into a clean, consistent, and machine-learning-ready format. What Did Our Raw Data Look Like? Remember our CSV files? Something like

Time Voltage

0.000 0 0.001 0.42 0.002 0.84 0.003 1.18. Question. Can SVM understand this directly? Technically, it expects feature vectors, not raw waveforms. So we must process it.

The Complete Preprocessing Pipeline

In our project, the preprocessing pipeline looks like this. **CSV Files** — Read using Python. Interpolation

Feature Extraction

Feature Matrix

Label Encoding

Train-Test Split

StandardScaler — Ready for Machine Learning. This is exactly what our code does. Notice

Machine Learning

still hasn't started.

Step 1 — Reading the CSV Files

Question. How does Python receive the LTspice data? Simple. We read the exported CSV files. In our code, this is where Pandas comes in. Example

```
df = pd.read_csv(file_path)
```

Question. What happens here? Python opens the CSV file and converts it into a DataFrame. Think of a DataFrame as an Excel sheet inside Python.

PROFESSOR ASKS

Why Pandas?

EXCELLENT ANSWER

Pandas provides efficient tools for reading, organizing, and processing structured tabular data such as CSV files.

Step 2 — Interpolation

Now one of the most important steps. Question. Suppose

Waveform A

contains 850 points.

Waveform B

contains 1023 points. Question. Can we compare them directly? No. Need Interpolation. Example

850 Points

Interpolation

1000 Points

Another waveform

1023 Points

Interpolation

1000 Points

Now both look identical in length. Question. Does interpolation create new information? No. It only estimates intermediate values to make every waveform have the same number of samples.

PROFESSOR ASKS

Why interpolation?

EXCELLENT ANSWER

Interpolation standardizes all waveforms to a common sampling representation, ensuring consistent feature extraction across the dataset. Why Didn't We Skip Interpolation? Suppose we skip it.

Waveform A

850 points.

Waveform B

1023 points. Now imagine calculating Rise Time. Question. Will the results be directly comparable? Not always. Interpolation makes feature extraction fair.

Step 3 — Feature Extraction

Now we have uniform waveforms. Question. Can SVM use 1000 voltage values? Maybe. But our project doesn't do that. Instead we calculate 17 features. Example Peak Mean RMS

Rise Time

Energy. DC Level These 17 numbers become one feature vector. Notice

Feature Extraction

is technically part of preprocessing. But because it is so important, we'll study it in a separate chapter.

Step 4 — Creating the Feature Matrix

Question. Suppose one waveform becomes [5.2, 3.4, 0.82,. 4.9] Now imagine 1000 waveforms. They become something like

Waveform 1

17 Features

Waveform 2

17 Features

Waveform 3

17 Features

. Together this forms the Feature Matrix. Think of it like an Excel table.

1	5.2	3.4	1.8	...
2	4.9	3.1	1.6	...
3	5.5	3.7	1.9	...

This is what

Machine Learning

actually receives.

PROFESSOR ASKS

What is a Feature Matrix?

EXCELLENT ANSWER

A feature matrix is a table in which each row represents one sample and each column represents one extracted feature.

Step 5 — Label Encoding

Question.

Machine Learning

understands numbers. Question. Can SVM understand RC_Open No. Need numbers. Example **RC_Normal** — 0. **RC_Open** — 1. **RL_Open** — 2. Now

Machine Learning

can understand the labels. Question. Does 0 mean better than 1? No. These are only identifiers.

PROFESSOR ASKS

Why Label Encoding?

EXCELLENT ANSWER

Machine learning algorithms require numerical target values. Label encoding converts textual class labels into numerical identifiers without changing their meaning.

Step 6 — Train-Test Split

This is one of the most important Machine Learning concepts. Question. Suppose you study the answers before an exam. Then during the exam, the teacher asks the same questions. Question. Can we judge your intelligence? No. Because you memorized them.

Machine Learning

has the same problem. So we split the dataset. Example

Entire Dataset

80% Training 20% Testing Training teaches the model. Testing checks whether it learned properly.

PROFESSOR ASKS
Why train-test split?
EXCELLENT ANSWER

The train-test split ensures that the model is evaluated on unseen data, providing a realistic estimate of its ability to generalize.

Step 7 — StandardScaler

Now the last preprocessing step. Suppose

Peak Voltage

is 500

Rise Time

is 0.001 Question. Which feature will dominate? Peak Voltage. Need Scaling. StandardScaler changes the scale without changing the information. Example Instead of 500 0.001 Both become values around 0 with similar spread. Now SVM treats every feature fairly.

PROFESSOR ASKS
Why StandardScaler?
EXCELLENT ANSWER

StandardScaler standardizes each feature so that all features contribute more equally during SVM training.

Complete Preprocessing Flow

Everything we have learned can be summarized using one diagram. **CSV Files** — Read using Pandas. Interpolation

Feature Extraction

Feature Matrix

Label Encoding

Train-Test Split

StandardScaler

Machine Learning

Congratulations. This is exactly what happens before SVM.

Connection to Our Project

This chapter is not theory. It is our actual implementation. Our Python code does these exact steps before training the classifier. When we study the code later, you'll recognize every single step.

COMMON MISTAKES

MISTAKE: [X] Preprocessing means only cleaning data. Wrong. It includes all transformations required before training.

MISTAKE: [X] Interpolation creates new information. Wrong. It estimates intermediate values.

MISTAKE: [X] Train-Test Split improves accuracy. Wrong. It improves evaluation reliability, not accuracy.

MISTAKE: [X] StandardScaler changes the meaning of the features. Wrong. It changes only the scale, not the information.

REVISION NOTES

Remember the preprocessing sequence. **CSV** — Interpolation. **Features** — Feature Matrix. **Label Encoding** — Train-Test Split. **StandardScaler** — Machine Learning. Memory Trick Remember: C → I → F → L → T → S Expand it. C = CSV I = Interpolation F = Features L = Label Encoding T = Train-Test Split S = StandardScaler Whenever someone asks "How did you preprocess the data?" just remember C → I → F → L → T → S

KNOWLEDGE CHECK

Level 1

- What is preprocessing?
- Why is interpolation necessary?
- What is a feature matrix?
- Why do we encode labels?
- Why do we use StandardScaler?

Level 2

- Why can't SVM use raw waveforms directly?
- Why is the train-test split important?
- Why are labels converted to numbers?
- What would happen if interpolation were skipped?
- Why is preprocessing critical for machine learning?

Professor Level

- Could your preprocessing pipeline be used with ANN without modification? Why?
- Why is preprocessing often considered more important than model selection?
- Which preprocessing step has the greatest impact on the quality of your feature extraction?
- What data leakage problems could occur if preprocessing is performed incorrectly?
- If you were given real oscilloscope measurements instead of LTspice data, which preprocessing steps would remain the same?
- End of Chapter 8

Congratulations. You now understand the complete preprocessing pipeline used in your project. More importantly, you now know why every step exists, not just what it does.

Author's Note (Important Improvement) Starting from the next chapter (Feature Extraction), we're entering the most technical part of your project. From that point onward, I'll begin integrating: Your actual code (line by line), The exact features used in your implementation, The reasoning behind each design decision, The improvements we made (like DC Level), And the presentation/viva explanations alongside the implementation. I believe the next few chapters (Feature Extraction, SVM, Evaluation, and ANN) will become the strongest part of the entire Project Bible because they are based directly on your implementation, not generic theory.